

Co to jest PLC i LD

Bartłomiej Kuk

February 9, 2026

1 Po co jest ten plik?

Plik powstaje po to, aby ułatwić pracę w projekcie osobom, które nie miały okazji pracować na sterowniku i nie wiedzą, co z czym się je.

2 Co to jest sterownik PLC

Sterownik PLC można nazwać komputerem przemysłowym, który odbiera sygnały z czujników i pozwala na dokonanie odpowiedniej akcji, np. czarne bloczki odrzucamy, a każdy inny jedzie dalej na linii produkcyjnej.

Jak działa program?

Program działa w nieskończonej pętli (Cyklu); dlatego trzeba umieć go programować i działać na różnych rodzajach zmiennych. Wykorzystuje się tutaj Program budowany jak drabinka; dlatego LD (ang. Ladder Diagram).

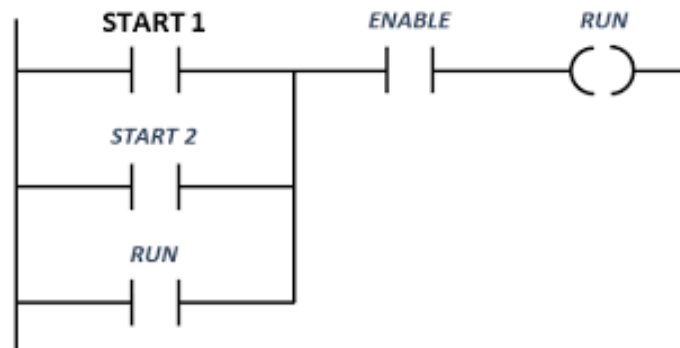


Figure 1: Przykładowy banalny program

3 Rodzaje zmiennych i adresowanie

W świecie PLC nie operujemy na nazwach zmiennych typu *moje_liczydło*, lecz na konkretnych obszarach pamięci sterownika. Każdy adres mówi nam, co to za sygnał i gdzie fizycznie (lub logicznie) się znajduje.

Obszary pamięci

- **I (Inputs) - Wejścia:** Sygnały płynące z zewnątrz do sterownika (np. przyciski, czujniki, fotokomórki).
- **Q (Outputs) - Wyjścia:** Sygnały sterujące elementami wykonawczymi (np. silniki, lampki, elektrozawory).
- **M (Memory) - Pamięć wewnętrzna:** Pamięć operacyjna sterownika. To nasz "brudnopis" do przechowywania stanów pośrednich, flag i danych obliczeniowych, których nie widać bezpośrednio na wyjściach fizycznych.

Typy danych i ich rozmiar

W zależności od tego, czy chcemy zapisać informację typu "tak/nie", czy dużą liczbę, musimy wybrać odpowiednią wielkość "szufladki" w pamięci:

Typ	Opis	Prefix	Przykład
BOOL	Typ bitowy (0 lub 1).	brak / X	I0.1, Q0.0, M1.2
BYTE	8 bitów (1 bajt).	B	MB10, IB2
WORD	16 bitów (Liczba całkowita).	W	MW100, QW4
DWORD	32 bity (Liczba zmiennoprzecinkowa/Real).	D	MD20, QD8

Table 1: Podstawowe typy danych w standardzie IEC 61131-3

Jak czytać adresy?

Adresowanie bitowe (najczęstsze w drabince) zapisujemy z kropką, np. **I1.5**:

- **I** – obszar (wejście),
- **1** – numer bajtu w pamięci,
- **5** – numer konkretnego bitu w tym bajcie (zakres 0-7).

Dla większych danych (liczby) używamy zapisu ciągłego, np. **MW10** – oznacza to 10-te "słowo" (Word) w pamięci *Memory*.

4 Logika wejść

Tak jak w matematyce, wejścia mogą podlegać operacjom logicznym. Podczas tworzenia ich należy myśleć, jak będzie płynął prąd przez dane styki (guziki).

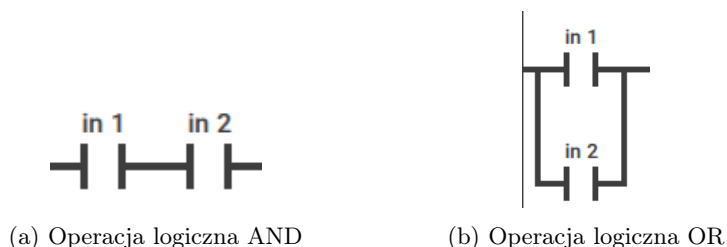
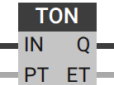
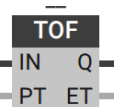
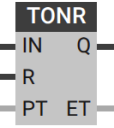


Figure 2: Porównanie operacji logicznych

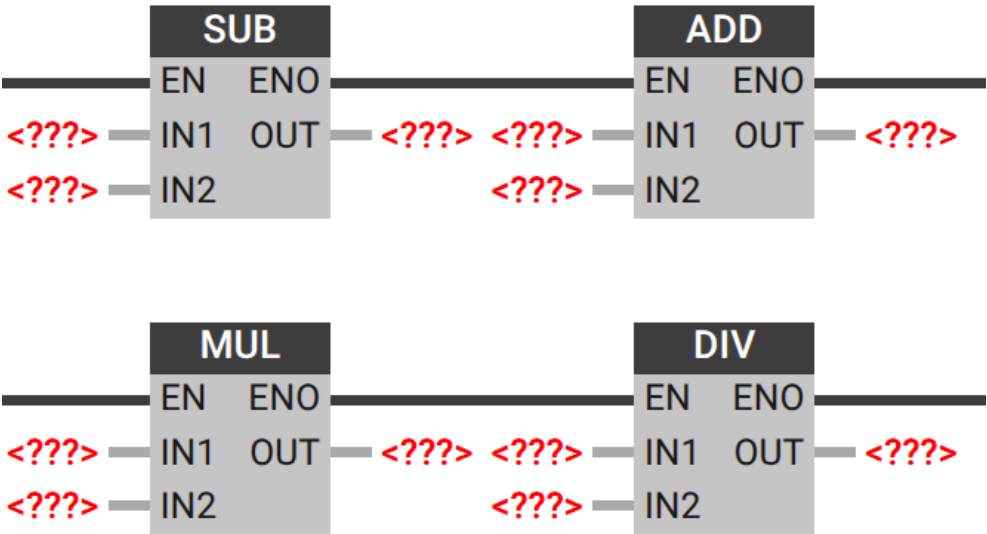
5 Opis elementów LD

W tabeli poniżej przedstawiono bloki, które należy zaimplementować.

Zdjęcie bloczku	Nazwa	Opis
	Styk normalnie otwarty <i>Normally Open Contact (NO)</i>	Można nazwać to guzikiem w ścianie. Przewodzi, gdy zmienna ma wartość 1.
	Styk normalnie zamknięty <i>Normally Closed Contact (NC)</i>	Przewodzi, gdy zmienna ma wartość 0 (negacja NO).
	Cewka <i>Coil</i>	Można nazwać żarówką, która działa tak długo jak jest wciśnięty guzik. Zapisuje wynik operacji logicznej do zmiennej wyjściowej.
	Cewka zanegowana <i>Negated Coil</i>	Zapisuje odwrotność wyniku operacji (NOT).
	Cewka Set <i>Set Coil</i>	Ustawia 1 na wyjściu tak długo jak tego nie zresetujemy.
	Cewka Reset <i>Reset Coil</i>	Resetujemy cewkę Set. Wymusza 0 na wyjściu.
TIMERY		Od teraz w opisie będzie opis dodatkowych wejść i wyjść,
	Timer TON <i>On-Delay Timer</i>	Opóźnione załączenie. Jak wciśniesz guzik (IN), żarówka zapali się dopiero po określonym czasie (PT). Jeśli puścisz guzik wcześniej – czas się zeruje i żarówka w ogóle nie mignie.
	Timer TOF <i>Off-Delay Timer</i>	Opóźnione wyłączenie. Żarówka zapala się od razu po wciśnięciu guzika, ale po jego puszczeniu świeci się jeszcze przez zadany czas (PT). Dobry przykład to światło na klatce schodowej.
	Timer TONR <i>Retentive On-Delay Timer</i>	Timer z pamięcią. Działa jak TON, ale jeśli puścisz guzik w trakcie odliczania, on nie zeruje czasu, tylko "pauzuje". Żeby zacząć od nowa, musisz użyć osobnego guzika Reset (R).

	MOVE	Przepisanie wartości. Kiedy dostanie sygnał (EN), bierze to co jest na wejściu (IN) i "przepycha" do wyjścia (OUT). Można to porównać do kopiowania pliku z jednego folderu do drugiego lub wartości przypisania = w programowaniu.
Liczniki		
	Licznik CTU <i>Count Up</i>	Licznik "w górę". Każdy impuls na wejściu (CU) zwiększa wynik o 1. Kiedy doliczy do zadanej wartości (PV), zapala "żarówkę" (Q). Jak wciśniesz (R), licznik zeruje się do zera.
	Licznik CTD <i>Count Down</i>	Licznik "w dół". Zaczyna od wysokiej wartości i odejmuje po 1 przy każdym kliknięciu (CD). Kiedy dobije do zera, zapala wyjście (Q). Wejście (LD) służy do ponownego załadowania wartości startowej.
	Licznik CTUD <i>Count Up-Down</i>	Kombajn. Może liczyć w górę (CU) i w dół (CD) jednocześnie. Ma dwa osobne wyjścia informujące, czy dobił do góry (QU), czy do dołu (QD). Idealny do liczenia aut na parkingu (wjeżdżające i wyjeżdżające).

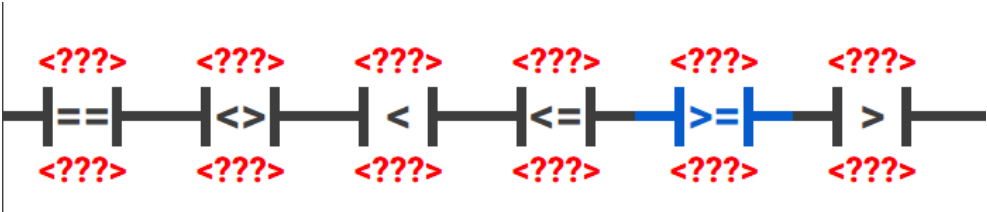
Oprócz operacji, które zostały wymienione powyżej, mamy również:



Rysunek 3: Operacje arytmetyczne

Gdzie każdy *IN** jest wejściem, a *OUT* jest wyjściem danej operacji algebraicznej. Można to traktować jak kalkulator, który sumuje lub odejmuje wartości i przesyła je dalej.

Natomiast do porównania, czy dane elementy są np. równe, czy różne itp., wykorzystuje się następujące bloki:



Rysunek 4: Operacje porównawcze

Działają one jak strażnicy bramki – puszczają sygnał dalej tylko wtedy, gdy warunek (np. czy licznik == 10) jest spełniony.